

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3069 – Programación Paralela y Distribuida

Sección 20

Ing. Santiago Solorzano



Proyecto 1 - ZipZip Espacial

Paralelización de una simulación gráfica con OpenMP

Ricardo Arturo Godínez Sánchez - 23247

Vianka Vanessa Castro Ordoñez - 23201

Abby Sofía Donis Agreda - 22440

GUATEMALA, 20 de agosto de 2026

Índice

Índice.....	2
Resumen	4
Introducción	4
Antecedentes	5
Computación paralela y memoria compartida.....	5
Modelo de ejecución de OpenMP	5
Rendimiento, speedup y escalabilidad	5
SDL2, OpenGL y separación de responsabilidades	6
Objetivos	6
Objetivo general	6
Objetivos específicos.....	6
Descripción y alcance del proyecto	7
Concepto de la simulación	7
Estado funcional y alcance incremental	7
Parámetros del problema	8
Diseño de la solución	8
Arquitectura del software.....	8
Estructuras de datos	8
Flujo secuencial de referencia	8
Identificación de regiones paralelizables	9
Aplicación del método PCAM	9
Diseño propuesto con OpenMP	9
Seguridad de concurrencia.....	10
Estado actual de implementación	10
Funciones completadas en la línea base	10
Pendientes técnicos antes de medir	11
Metodología experimental.....	11
Preguntas e hipótesis	11
Variantes a comparar	12

Métricas	12
Factores y configuraciones	12
Procedimiento de medición	13
Validación de corrección.....	13
Amenazas a la validez	14
Resultados	14
Entorno de ejecución	14
Corrección funcional.....	14
Tiempos de actualización.....	14
Tiempo total y FPS.....	15
Comparación de scheduling	15
Análisis y discusión	15
Conclusiones	16
Recomendaciones	16
Apéndices	16
Apéndice A. Registro de hardware y software.....	16
Apéndice B. Parámetros y controles	17
Apéndice C. Pseudocódigo de benchmark	17
Apéndice D. Lista de verificación para la entrega.....	17
Bibliografía.....	18

Resumen

ZipZip Espacial es un screensaver interactivo desarrollado en C++ que representa una granja suspendida en el espacio. La versión base utiliza SDL2 para crear la ventana y administrar eventos, OpenGL para el renderizado y estructuras de datos propias para actualizar vacas y estrellas. El propósito del proyecto es construir primero una referencia secuencial correcta y, a partir de ella, paralelizar con OpenMP las actualizaciones independientes de los elementos de la escena. La evaluación propuesta separa el costo de simulación del costo de renderizado y compara tiempo de ejecución, cuadros por segundo, speedup y eficiencia para diferentes tamaños del problema y cantidades de hilos. El presente informe documenta el diseño, el estado actual y el protocolo de medición; los resultados cuantitativos y las conclusiones se incorporarán cuando finalice la implementación paralela y se ejecuten todas las configuraciones experimentales.

Introducción

El aumento en la cantidad de núcleos disponibles en los procesadores actuales permite reducir el tiempo de ciertas tareas cuando el trabajo puede dividirse en operaciones independientes. Sin embargo, el simple hecho de agregar hilos no garantiza una mejora: la porción secuencial, la sincronización, el movimiento de datos y la granularidad del trabajo limitan la aceleración alcanzable. Por esta razón, un proyecto de computación paralela debe comparar una línea base bien definida con una versión paralela funcionalmente equivalente, y medir ambas bajo condiciones controladas.

En este proyecto se utiliza una simulación gráfica como caso de estudio. La escena contiene múltiples vacas con posición, velocidad, giro, escala y color propios, además de un campo de estrellas con movimiento y variación de brillo. Cada elemento actualiza principalmente su propio estado, de modo que los ciclos de actualización exhiben paralelismo de datos. OpenMP resulta apropiado porque permite expresar ese paralelismo sobre memoria compartida mediante directivas de compilador y rutinas de ejecución, sin reemplazar la organización general del programa en C++ (Dagum y Menon, 1998; OpenMP Architecture Review Board, 2021).

La aplicación también plantea una separación importante entre simulación y presentación. Las transformaciones y reglas de movimiento se calculan en CPU, mientras que el dibujo se realiza por medio de OpenGL. Las operaciones de video de SDL2 deben permanecer en el hilo principal en la mayoría de los backends, por lo que la estrategia del proyecto consiste en paralelizar la actualización de datos y conservar el manejo de eventos y el renderizado en el hilo principal (SDL contributors, s. f.). Esta decisión reduce el riesgo de condiciones de carrera sobre el contexto gráfico y permite atribuir las mejoras al trabajo de simulación.

El alcance de este informe es presentar una estructura completa y reproducible antes de cerrar la fase experimental. Se describen los antecedentes, los objetivos, la arquitectura actual, las regiones candidatas para OpenMP, las variables compartidas y privadas, el método de medición y los formatos que se utilizarán para reportar resultados. No se incluyen cifras de

rendimiento inventadas: las tablas y figuras de resultados quedan preparadas para llenarse con datos obtenidos en las máquinas del equipo.

Antecedentes

Computación paralela y memoria compartida

Flynn (1972) clasificó las arquitecturas de cómputo según la relación entre flujos de instrucciones y flujos de datos. Los procesadores multinúcleo utilizados en este proyecto permiten que varios hilos ejecuten instrucciones sobre distintos elementos de una misma colección, lo cual se aproxima al patrón MIMD a nivel de núcleos y puede complementarse con vectorización dentro de cada núcleo. En el modelo de memoria compartida, los hilos acceden al mismo espacio de direcciones. Esto simplifica el intercambio de información, pero exige definir con claridad qué datos se comparten, cuáles son privados y en qué puntos se requiere sincronización.

Las simulaciones con numerosas entidades suelen actualizar atributos equivalentes para cada elemento. Si una entidad no necesita leer ni modificar el estado de las demás durante una iteración, el conjunto puede partitionarse entre hilos. Este patrón aparece en animaciones y modelos de agentes, donde cada entidad mantiene un estado local y aplica reglas de movimiento en pasos discretos (Reynolds, 1987). En ZipZip Espacial, las vacas rebotan contra límites globales y las estrellas se envuelven al cruzar el borde; ambas operaciones pueden evaluarse por elemento sin actualizar posiciones ajenas.

Modelo de ejecución de OpenMP

OpenMP sigue un modelo fork-join. El programa inicia con un hilo principal; al entrar a una región paralela se crea un equipo de hilos y, al terminar la región, la ejecución vuelve a un solo hilo. El constructo de distribución de ciclos permite repartir iteraciones entre el equipo. Para un ciclo con costo uniforme por elemento, `schedule(static)` suele ser un punto de partida razonable porque asigna bloques de iteraciones con poco overhead. Si el costo varía de manera importante entre entidades, `schedule(dynamic)` puede mejorar el balance de carga, aunque introduce más coordinación. La decisión debe basarse en mediciones y no únicamente en intuición.

La especificación de OpenMP también define reglas de compartición de datos y rutinas como `omp_get_wtime` para medición de tiempo. En el diseño propuesto, los vectores que contienen las entidades son compartidos, pero cada iteración obtiene una referencia a un elemento distinto. Las variables temporales del cuerpo del ciclo son privadas; el paso de tiempo y los límites de la escena son valores de solo lectura. El uso de `default(none)` en las regiones nuevas se recomienda porque obliga a declarar la intención de cada variable y ayuda a detectar errores.

Rendimiento, speedup y escalabilidad

La Ley de Amdahl establece que la aceleración de un programa está limitada por la fracción que permanece secuencial (Amdahl, 1967). En esta aplicación, el procesamiento de eventos, las llamadas de renderizado y el intercambio de buffers continuarán en el hilo principal; por

tanto, aunque la actualización de miles de entidades se acelere, el tiempo total por cuadro no necesariamente disminuirá en la misma proporción. La Ley de Gustafson complementa esta perspectiva al considerar problemas cuyo tamaño aumenta con los recursos disponibles (Gustafson, 1988). Esto justifica evaluar escenas pequeñas y grandes: OpenMP puede no compensar su overhead con 20 vacas, pero sí con miles de elementos.

El rendimiento también depende del acceso a memoria y de la capacidad de cómputo del procesador. El modelo Roofline relaciona intensidad aritmética, ancho de banda de memoria y rendimiento máximo, y recuerda que una rutina con pocas operaciones por dato puede quedar limitada por memoria antes de ocupar todos los núcleos (Williams, Waterman y Patterson, 2009). Por ello, además del tiempo total, conviene medir únicamente las funciones de actualización y observar si el beneficio se estabiliza al aumentar los hilos.

SDL2, OpenGL y separación de responsabilidades

SDL2 administra la ventana, el teclado y el contexto de OpenGL. OpenGL define el proceso de transformación y rasterización utilizado para dibujar los modelos, y su especificación constituye la referencia normativa de la API gráfica (Khronos Group, 2022). El código actual usa un contexto OpenGL asociado a la ventana de SDL2, arreglos de vértices y normales para el modelo OBJ, y llamadas de dibujo por cada vaca. Esta parte no será distribuida con OpenMP durante el proyecto, porque comparte estado del contexto y porque el objetivo central es analizar el paralelismo en CPU.

Mantener el renderizado en el hilo principal también mejora la interpretación de los resultados. El tiempo de actualización representa el trabajo que OpenMP sí puede acelerar, mientras que el tiempo de cuadro completo incluye actualización, dibujo, manejo de eventos y espera por sincronización vertical. Se reportarán ambas mediciones para no confundir una mejora local con una mejora visible en toda la aplicación.

Objetivos

Objetivo general

Diseñar, implementar y evaluar una versión paralela con OpenMP de la actualización de entidades de ZipZip Espacial, comparándola con una versión secuencial equivalente mediante métricas de tiempo, FPS, speedup y eficiencia bajo diferentes tamaños de escena y cantidades de hilos.

Objetivos específicos

- Conservar una versión secuencial determinista que funcione como referencia de corrección y rendimiento.
- Identificar ciclos y tareas independientes en la actualización de vacas, estrellas y futuros sistemas de partículas.
- Aplicar directivas de OpenMP con una política explícita de variables shared, private y firstprivate, evitando condiciones de carrera.

- Mantener las operaciones de SDL2 y OpenGL en el hilo principal para proteger el contexto gráfico.
- Comparar al menos una estrategia de scheduling estático con alternativas justificadas por el costo de las iteraciones.
- Medir cada configuración en múltiples repeticiones y reportar variación, speedup, eficiencia y comportamiento al escalar el problema.
- Documentar el hardware, compilador, opciones de optimización, semillas y parámetros necesarios para reproducir los experimentos.

Descripción y alcance del proyecto

Concepto de la simulación

ZipZip Espacial representa una granja nocturna suspendida en un entorno espacial. Las vacas se desplazan dentro del área visible, giran y rebotan contra sus límites. Un fondo de estrellas se mueve lentamente y cambia de brillo. La propuesta artística contempla además un OVNI pilotado por un gato, capturas de vacas, partículas, planetas y eventos visuales; estos elementos se consideran alcance incremental y deben distinguirse de las funciones ya presentes en la versión base.

Estado funcional y alcance incremental

Subsistema	Estado de la línea base	Meta del proyecto
Ventana y eventos	Ventana de 900 x 700, teclado, pausa y cierre	Conservar en el hilo principal
Vacas	Cantidad configurable; posición, velocidad, giro, escala y color	Paralelizar la actualización por entidad
Estrellas	180 por defecto; movimiento, envoltura y brillo	Paralelizar la actualización por entidad
Modelo 3D	Carga de OBJ, triangulación, normalización y normales	Mantener carga fuera del ciclo medido
Renderizado	OpenGL, iluminación, wireframe, culling y HUD	Mantener secuencial; medir su impacto
OVNI y partículas	Previstos en la propuesta, no integrados en la línea base revisada	Agregar solo si se completa y valida antes de medir
Rendimiento	FPS visible; VSync conmutable	Agregar temporización reproducible y exportación de datos

Parámetros del problema

Para describir los experimentos se utilizarán símbolos consistentes. N representa la cantidad de vacas; M, la cantidad de estrellas; K, la capacidad o cantidad activa de partículas cuando exista ese sistema; P, la cantidad de elementos decorativos o planetas si su actualización forma parte del costo; T, el número de hilos de OpenMP; W y H, el ancho y alto de la ventana; y seed, la semilla de generación pseudoaleatoria. El informe final debe indicar cuáles parámetros afectan la simulación y cuáles afectan únicamente el dibujo.

Diseño de la solución

Arquitectura del software

El repositorio separa responsabilidades en módulos pequeños. main.cpp configura SDL2 y OpenGL, procesa eventos, calcula el paso de tiempo y coordina el ciclo principal. El módulo scene administra las instancias de vacas; starfield administra las estrellas; renderer concentra las llamadas de OpenGL y el HUD; y obj_loader carga la geometría Wavefront OBJ sin depender de SDL2 ni del renderizador. CMake configura el proyecto en C++17, enlaza SDL2 y OpenGL y copia los recursos al directorio del ejecutable.

- Aplicación: inicialización, argumentos, eventos, pausa, VSync y ciclo principal.
- Simulación: creación y actualización del estado de vacas y estrellas.
- Renderizado: proyección, iluminación, dibujo de estrellas y vacas, y HUD.
- Recursos: lectura, triangulación y normalización del modelo OBJ.
- Rendimiento: temporización de la línea base y de la variante OpenMP, aún por integrar como módulo explícito.

Estructuras de datos

Cada Instancia de vaca almacena posición x-y, ángulo y velocidad de giro, escala, color y velocidad lineal. La estructura Escena contiene un `std::vector<Instancia>` y los límites visibles. Cada Estrella mantiene posición, velocidad, tamaño y parámetros de brillo; CampoEstrellas contiene el vector, los límites y un tiempo global. El modelo OBJ se transforma en arreglos contiguos de posiciones, normales y coordenadas de textura. El uso de vectores facilita distribuir rangos de índices entre hilos, aunque debe vigilarse que ninguna región paralela cambie el tamaño del vector mientras otros hilos conservan referencias a sus elementos.

Flujo secuencial de referencia

1. Leer argumentos, inicializar SDL2 y crear la ventana y el contexto OpenGL.
2. Cargar y normalizar el modelo de vaca; preparar los arreglos de vértices y normales.
3. Crear la escena y el campo de estrellas con semillas fijas.

4. Procesar eventos y calcular Δt en cada cuadro, limitándolo para evitar saltos extremos.
5. Si la simulación no está pausada, actualizar todas las vacas y después todas las estrellas.
6. Dibujar el fondo, las vacas y el HUD; finalmente intercambiar los buffers de la ventana.
7. Al cerrar, liberar el estado de OpenGL y los recursos de SDL2.

Identificación de regiones paralelizables

El candidato principal es el ciclo de actualizarEscena. En una iteración se modifica únicamente `e.vacas[i]`; los límites y Δt son valores de solo lectura. El segundo candidato es el ciclo de actualizarCampoEstrellas, en el cual cada iteración modifica una estrella diferente. El incremento de `campo.tiempo` debe ejecutarse una sola vez antes del ciclo paralelo. Si se incorpora un sistema de partículas, su actualización puede seguir el mismo patrón siempre que la creación y eliminación de partículas no cambie concurrentemente el contenedor.

No se propone paralelizar la lectura de eventos, la modificación del título de la ventana, la carga inicial del modelo ni las llamadas de dibujo. Son tareas de baja frecuencia o asociadas a estado externo. Esta división mantiene una frontera clara: los hilos de trabajo producen un estado de simulación completo y el hilo principal consume ese estado para dibujar el cuadro.

Aplicación del método PCAM

Partición. Separar los arreglos de vacas, estrellas y partículas en rangos de índices. Cada entidad constituye una unidad de trabajo independiente durante un paso de simulación.

Comunicación. Las iteraciones no necesitan intercambiar datos entre sí. Comparten Δt , límites y tiempo global como valores de lectura; la comunicación ocurre al finalizar la región, cuando el hilo principal utiliza los estados actualizados.

Aglomeración. Agrupar múltiples entidades por hilo para que el trabajo útil compense el costo de iniciar y sincronizar una región paralela. Para escenas pequeñas puede ser preferible ejecutar secuencialmente mediante una condición `if` en la directiva.

Mapeo. Asignar rangos contiguos con `schedule(static)` como configuración inicial. Las políticas `dynamic` o `guided` se evaluarán solamente si la carga por entidad deja de ser uniforme.

Diseño propuesto con OpenMP

La opción más directa es aplicar `parallel for` a cada ciclo de actualización. Para reducir el costo de crear equipos dos veces por cuadro, una segunda variante puede utilizar una sola región `parallel` y dos constructos `for`. El uso de `nowait` entre los ciclos es válido únicamente si las estrellas no dependen de las vacas y si ninguna tarea intermedia consume resultados parciales. Debe existir una barrera antes de dibujar, ya que el renderizador requiere que todos los elementos hayan terminado su actualización.

```
// Patrón propuesto; se integrará y validará en el código fuente.
#pragma omp parallel default(none) shared(escena, campo) firstprivate(dt)
{
    #pragma omp for schedule(static) nowait
    for (size_t i = 0; i < escena.vacas.size(); ++i) { /* actualizar vaca i */ }

    #pragma omp for schedule(static)
    for (size_t i = 0; i < campo.estrellas.size(); ++i) { /* actualizar estrella i */ }
}
}
```

El fragmento anterior es una guía de diseño, no evidencia de que la directiva ya esté integrada. Antes de adoptarlo se debe mover la actualización de `campo.tiempo` fuera del segundo ciclo, comprobar la compartición de variables y verificar que la versión paralela produzca los mismos estados que la secuencial para una semilla y un `dt` fijos.

Seguridad de concurrencia

Dato o recurso	Tratamiento	Justificación
dt y límites	firstprivate o shared de solo lectura	No cambian dentro de la región
Vectores de entidades	shared	Cada iteración modifica un índice distinto
Referencia local a una entidad	private	Se crea dentro de cada iteración
campo.tiempo	Actualización única antes del for	Evita una carrera sobre el acumulador global
Generador pseudoaleatorio	Secuencial al crear; estado por hilo si se usa en paralelo	Un RNG compartido produciría carreras y resultados no deterministas
SDL2 y contexto OpenGL	Solo hilo principal	El backend de video y el contexto gráfico mantienen estado externo
Contadores globales	reduction o atomic, si se agregan	Evita actualizaciones perdidas

Estado actual de implementación

Funciones completadas en la línea base

- Creación de una ventana OpenGL de 900 x 700 mediante SDL2.
- Carga de un modelo Wavefront OBJ, triangulación en abanico y generación de normales cuando faltan.
- Normalización del modelo y almacenamiento de posiciones y normales en arreglos contiguos.
- Creación determinista de vacas y estrellas con generadores xorshift32 y semillas separadas.

- Movimiento, giro, rebote de vacas, envoltura de estrellas y variación sinusoidal de brillo.
- Renderizado con iluminación, control de wireframe, face culling y VSync.
- HUD con FPS, cantidad de vacas y cantidad de estrellas.
- Organización del proyecto con CMake, C++17, SDL2 y OpenGL.

Pendientes técnicos antes de medir

- Agregar OpenMP al sistema de construcción y producir binarios serial y paralelo comparables.
- Separar temporizadores de actualización, renderizado y cuadro completo.
- Permitir configurar N, M, T y seed sin recompilar; documentar valores predeterminados.
- Registrar resultados en CSV o en un formato tabular para evitar transcripción manual.
- Agregar un modo de benchmark sin renderizado para aislar el costo computacional de la simulación.
- Validar equivalencia entre variantes con semilla y paso de tiempo fijos.
- Decidir si OVNI, partículas y planetas se incorporarán antes de congelar el alcance experimental.

Metodología experimental

Preguntas e hipótesis

Pregunta 1. ¿A partir de qué tamaño de escena el trabajo útil compensa el overhead de OpenMP?

Pregunta 2. ¿Cómo cambia el speedup cuando se aumenta T desde un hilo hasta la cantidad de hilos lógicos disponibles?

Pregunta 3. ¿La mejora de la actualización se refleja en el tiempo total por cuadro o el renderizado se convierte en el cuello de botella?

Hipótesis. La versión paralela tendrá poco o ningún beneficio en escenas pequeñas, pero reducirá el tiempo de actualización en escenas grandes. El speedup del cuadro completo será menor que el speedup de la actualización porque SDL2 y OpenGL permanecen secuenciales. Para el estado actual, schedule(static) debería producir menor overhead que dynamic debido a que cada entidad ejecuta una cantidad similar de operaciones.

Variantes a comparar

Variante	Descripción	Propósito
SEQ	Compilación sin directivas OpenMP activas	Línea base funcional y de tiempo
OMP-1	Mismo código OpenMP ejecutado con T = 1	Estimar overhead del runtime
OMP-p	OpenMP con T = 2, 4, 8, ... hasta el límite del equipo	Medir escalabilidad
STATIC	Distribución estática de iteraciones	Caso principal para carga uniforme
DYNAMIC	Distribución dinámica con chunk documentado	Comprobar costo y balance si la carga cambia

Métricas

Sea T1 el tiempo de la versión secuencial para una configuración y Tp el tiempo de la versión paralela con p hilos. El speedup indica cuántas veces es más rápida la ejecución paralela; la eficiencia expresa qué fracción del paralelismo ideal se aprovecha. La mejora porcentual facilita comunicar la reducción de tiempo. Las tres métricas se calcularán tanto para actualización como para el tiempo total cuando corresponda.

$$\text{Speedup: } S(p) = T1 / Tp$$

$$\text{Eficiencia: } E(p) = [S(p) / p] \times 100 \%$$

$$\text{Mejora: } I(p) = [(T1 - Tp) / T1] \times 100 \%$$

$$\text{FPS promedio} = \text{cuadros renderizados} / \text{tiempo observado}$$

Se reportarán media, mediana, mínimo, máximo y desviación estándar de las repeticiones. La mediana será la medida principal cuando existan valores atípicos causados por procesos del sistema. No se mezclará el FPS limitado por VSync con el FPS libre; para benchmarking se usará VSync desactivado y se registrará cualquier límite adicional del controlador gráfico.

Factores y configuraciones

Factor	Valores iniciales propuestos	Control
N: vacas	20, 100, 500, 1 000, 5 000	Misma semilla y modelo
M: estrellas	180, 1 000, 5 000, 20 000	Separar pruebas de N y M
T: hilos	1, 2, 4, 8, ... hasta hilos lógicos	No exceder el límite sin justificar
Scheduling	static; dynamic con chunk documentado	Mismo trabajo y compilación
Repeticiones	Mínimo 10 por configuración	Mismo número para todas
VSynC	Desactivado en pruebas de FPS	Confirmar valor en cada sesión
Optimización	Release, por ejemplo -O2 o /O2	Mismas banderas en SEQ y OMP

Los valores son una matriz inicial y pueden reducirse si el renderizado se vuelve impráctico. Para aislar OpenMP debe existir una prueba de actualización sin llamadas de dibujo, con una cantidad de pasos fija. La prueba interactiva se conserva para FPS y experiencia visual, pero no sustituye la medición aislada del kernel de simulación.

Procedimiento de medición

1. Registrar procesador, núcleos físicos, hilos lógicos, memoria, sistema operativo, compilador y versión de OpenMP.
2. Cerrar aplicaciones no necesarias, conectar el equipo a energía y conservar el mismo perfil de rendimiento.
3. Compilar SEQ y OMP en modo Release con las mismas opciones, excepto las necesarias para OpenMP.
4. Ejecutar una fase de calentamiento que no se incluya en los resultados.
5. Fijar N, M, seed, cantidad de pasos y T; desactivar VSync para las pruebas de FPS.
6. Medir primero el kernel de actualización con `omp_get_wtime` y después el cuadro completo.
7. Ejecutar al menos diez repeticiones por configuración, alternando el orden de variantes cuando sea posible.
8. Guardar cada observación individual, no solo el promedio, y conservar fecha y equipo de origen.
9. Calcular estadísticas, speedup y eficiencia mediante una hoja o script reproducible.
10. Repetir cualquier dato atípico únicamente si existe una causa documentada; no descartar resultados de manera selectiva.

Validación de corrección

La versión paralela debe evaluarse primero con un dt fijo y una semilla fija. Después de una cantidad conocida de pasos se compararán posiciones, velocidades, giros y brillo con la versión secuencial. Debido a operaciones de punto flotante, se utilizará una tolerancia documentada, por ejemplo 10^{-5} para atributos que siguen el mismo orden de operaciones. También se comprobará que todas las entidades permanezcan dentro de los límites, que no existan valores NaN y que el programa cierre sin errores para cada T.

Si el sistema incorpora reducciones, decisiones globales o generación aleatoria dentro de regiones paralelas, la comparación debe ampliarse. Una ejecución visualmente correcta no demuestra ausencia de carreras; se recomienda activar advertencias del compilador y, si el entorno lo permite, utilizar un detector de carreras o pruebas repetidas con diferentes cantidades de hilos.

Amenazas a la validez

- Sincronización vertical o límites del controlador que oculten diferencias reales de tiempo.
- Frecuencia dinámica, temperatura y ahorro de energía que cambien durante las pruebas.
- Procesos del sistema operativo y aplicaciones en segundo plano.
- Comparación de binarios con opciones de compilación distintas.
- Escenas demasiado pequeñas, en las que el overhead domina, o demasiado grandes, en las que el renderizado domina.
- Uso de una sola computadora, que limita la generalización a otras arquitecturas.
- Medición del cuadro completo sin separar simulación y presentación.
- Cambios funcionales entre SEQ y OMP que vuelvan injusta la comparación.

Resultados

Entorno de ejecución

Espacio reservado para el entorno. Completar una fila por computadora con la plantilla del Apéndice A. Indicar cuál equipo se utilizará para las gráficas principales y conservar los resultados de los demás como replicación o comparación secundaria.

Equipo	CPU / núcleos / hilos	RAM	SO	Compilador	OpenMP
A	Por registrar	Por registrar	Por registrar	Por registrar	Por registrar
B	Por registrar	Por registrar	Por registrar	Por registrar	Por registrar
C	Por registrar	Por registrar	Por registrar	Por registrar	Por registrar

Corrección funcional

Espacio reservado para validación. Reportar el número de pasos, dt, seed, tolerancia máxima observada y resultado de las pruebas de límites. Incluir cualquier diferencia entre SEQ y OMP antes de presentar rendimiento.

Tiempos de actualización

N / M	T	Mediana Tp (ms)	Desv. est.	Speedup	Eficiencia
Por definir	1	Pendiente	Pendiente	1.00	100 %
Por definir	2	Pendiente	Pendiente	Pendiente	Pendiente

N / M	T	Mediana Tp (ms)	Desv. est.	Speedup	Eficiencia
Por definir	4	Pendiente	Pendiente	Pendiente	Pendiente
Por definir	8	Pendiente	Pendiente	Pendiente	Pendiente

Figura 1 por insertar. Gráfica de tiempo de actualización contra cantidad de hilos, con una serie por tamaño del problema. Incluir unidades, leyenda y barras de error.

Figura 2 por insertar. Gráfica de speedup contra cantidad de hilos. Agregar una línea ideal $S(p) = p$ para facilitar la comparación.

Tiempo total y FPS

Configuración	SEQ FPS	OMP FPS	Mejora FPS	Tiempo cuadro SEQ	Tiempo cuadro OMP
Escena pequeña	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente
Escena mediana	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente
Escena grande	Pendiente	Pendiente	Pendiente	Pendiente	Pendiente

Figura 3 por insertar. Comparación de FPS o tiempo por cuadro entre SEQ y la mejor configuración OMP. Indicar explícitamente que VSync estuvo desactivado.

Comparación de scheduling

T	static	dynamic (chunk)	guided	Mejor opción	Observación
2	Pendiente	Pendiente	Opcional	Pendiente	
4	Pendiente	Pendiente	Opcional	Pendiente	
8	Pendiente	Pendiente	Opcional	Pendiente	

Análisis y discusión

Espacio reservado para la discusión. Interpretar los resultados sin limitarse a repetir tablas. Explicar el punto en que OpenMP comienza a ser útil, el T con mejor compromiso, la caída de eficiencia, la diferencia entre tiempo de actualización y tiempo total, y el efecto del scheduling. Relacionar los hallazgos con la Ley de Amdahl, el tamaño del problema, el overhead y las características del hardware.

Preguntas que debe responder el análisis. ¿La hipótesis se sostuvo? ¿El resultado fue consistente entre repeticiones y equipos? ¿El renderizado limitó el speedup visible? ¿Hubo una configuración con más hilos pero peor tiempo? ¿Qué parte del código debería optimizarse después?

Conclusiones

Espacio reservado para conclusiones finales. Redactar de tres a cinco conclusiones basadas directamente en datos. Cada conclusión debe indicar el tamaño del problema y la configuración que la respalda. Evitar afirmar que OpenMP siempre mejora el rendimiento o presentar como conclusión una característica de diseño que no fue medida.

Recomendaciones

Antes de ejecutar la campaña definitiva se recomienda congelar el alcance funcional y conservar dos binarios comparables. Las mediciones de actualización deben ser independientes del renderizado, mientras que el FPS se utiliza como métrica complementaria de la experiencia completa. También se recomienda iniciar con `schedule(static)`, usar `default(none)`, mantener la generación aleatoria fuera de regiones compartidas y evitar cambiar el tamaño de los vectores mientras se actualizan en paralelo.

Si los resultados muestran que el renderizado domina, la siguiente etapa no debería consistir en agregar más hilos de CPU a los mismos ciclos, sino en reducir llamadas de dibujo, utilizar instancing o modernizar la transferencia de geometría. Esa optimización pertenece a una fase posterior y debe reportarse por separado para no atribuir a OpenMP mejoras que provengan de cambios en OpenGL.

Espacio reservado para recomendaciones derivadas de resultados. Agregar recomendaciones específicas sobre *T*, scheduling, tamaño mínimo útil y configuración de hardware una vez finalizado el análisis.

Apéndices

Apéndice A. Registro de hardware y software

Campo	Valor por completar
Integrante / identificador del equipo	
Procesador y frecuencia reportada	
Núcleos físicos / hilos lógicos	
Memoria RAM	
GPU y controlador	
Sistema operativo y versión	
Compilador y versión	
Versión o macro <code>_OPENMP</code>	
Opciones de compilación	

Campo	Valor por completar
Perfil de energía	
Fecha y hora de la prueba	

Apéndice B. Parámetros y controles

Entrada o control	Uso
Primer argumento numérico	Cantidad de vacas N; el valor predeterminado actual es 20
Ruta no numérica	Ruta al modelo OBJ; por defecto assets/models/cow.obj
Espacio	Pausar o continuar la simulación
W	Alternar entre relleno y wireframe
C	Activar o desactivar face culling
V	Activar o desactivar VSync
Esc	Cerrar la aplicación
OMP_NUM_THREADS o parámetro T	Fijar la cantidad de hilos de OpenMP
seed	Reproducir la distribución inicial; debe exponerse antes de medir

Apéndice C. Pseudocódigo de benchmark

```

crear_estado(N, M, seed)
repetir calentamiento:
    actualizar(dt)
inicio = omp_get_wtime()
repetir pasos_medidos:
    actualizar(dt)
fin = omp_get_wtime()
tiempo_por_paso = (fin - inicio) / pasos_medidos
guardar(N, M, T, scheduling, repetición, tiempo_por_paso)

```

El estado debe reconstruirse con la misma semilla antes de cada repetición o restaurarse desde una copia equivalente. El temporizador no debe incluir carga del modelo, creación de ventana, escritura de archivos ni calentamiento. Para el benchmark de cuadro completo se utiliza un bloque separado que sí incorpora actualización y renderizado, pero mantiene fuera la inicialización.

Apéndice D. Lista de verificación para la entrega

- El código fuente compila desde cero siguiendo las instrucciones del README.
- SEQ y OMP producen resultados equivalentes para una prueba determinista.

- El repositorio identifica claramente la directiva y las variables compartidas y privadas.
- Las tablas contienen unidades, repeticiones y estadística descriptiva.
- Las gráficas tienen título, ejes, leyenda y referencia en el texto.
- Las conclusiones citan valores concretos y no repiten únicamente la metodología.
- Se incluyen hardware, compilador, opciones de optimización, T, scheduling, N, M y seed.
- No se incluyen fuentes inventadas ni documentos proporcionados por el curso como bibliografía externa.
- El índice y la numeración de páginas fueron actualizados antes de entregar.

Bibliografía

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. En AFIPS Spring Joint Computer Conference, 30, 483-485.
<https://doi.org/10.1145/1465482.1465560>
- Dagum, L., y Menon, R. (1998). OpenMP: An industry standard API for shared-memory programming. IEEE Computational Science & Engineering, 5(1), 46-55. <https://doi.org/10.1109/99.660313>
- Flynn, M. J. (1972). Some computer organizations and their effectiveness. IEEE Transactions on Computers, C-21(9), 948-960. <https://doi.org/10.1109/TC.1972.5009071>
- Gustafson, J. L. (1988). Reevaluating Amdahl's Law. Communications of the ACM, 31(5), 532-533.
<https://doi.org/10.1145/42411.42415>
- Khronos Group. (2022). OpenGL 4.6 Core Profile Specification.
<https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.pdf>
- OpenMP Architecture Review Board. (2021). OpenMP Application Programming Interface, Version 5.2. <https://www.openmp.org/spec-html/5.2/openmp.html>
- Reynolds, C. W. (1987). Flocks, herds and schools: A distributed behavioral model. ACM SIGGRAPH Computer Graphics, 21(4), 25-34. <https://doi.org/10.1145/37402.37406>
- SDL contributors. (s. f.). FAQ: Development. SDL2 Wiki. Recuperado el 20 de agosto de 2026 de <https://wiki.libsdl.org/SDL2/FAQDevelopment>
- Williams, S., Waterman, A., y Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65-76.
<https://doi.org/10.1145/1498765.1498785>